Exploring Autonomous Optimization: Experiments with AlphaEvolve and LLM Economics

Anthropic revealed a hiring challenge they gave to applicants as a take home interview Assignment a few days ago that I have been looking into https://github.com/anthropics/original_performance_takehome/tree/main. Without getting to deep into it It's basically a code optimization problem where you're trying to get the most out of a processor that has a lot of parallel capacity. The computation is fixed. You just need to structure the code so the hardware is actually being utilized instead of mostly sitting idle. A good optimization completes the computation in as few clock cycles as possible.

They revealed that when they gave Opus 4.5 the challenge, it was able to complete it better than the applicants that applied. Every time they let Opus think about it further, it did better and better, getting close to the most optimal solution. At the most extreme, Claude Opus 4.5 after 11.5 hours reached 1,487 cycles. That being said, the best human solutions were revealed to be significantly better than that. But in their challenge they mentioned that if you beat Opus's best performance, you should send them an email and a resume.

This challenge aims to recruit very experienced Compiler Engineers, Performance Engineers, or Systems Architects. Someone who works on embedded systems, GPUs, or high-performance computing has this skillset needed to find a good solution. It's less about algorithms and more about understanding how hardware actually executes code and how to keep it fed with useful work.

This not being me, I decided to approach the challenge in a different way. The problem was setup perfectly to fit into a familiar framework that I read about a few months ago. This was AlphaEvolve by Google DeepMind. I will give a quick explanation but I encourage you to look into further if interested. The paper is a very good read. (https://arxiv.org/pdf/2506.13131)

AlphaEvolve is Google DeepMind's evolutionary coding agent infrastructure. Instead of asking an LLM to solve a problem once, you let it try thousands of times, each attempt building on what worked before. An LLM proposes a change to some code. The system automatically tests if that change actually made things better. If it did, that version survives. If not, it gets thrown away. The best solutions become parents for the next generation. Natural selection, but for algorithms.

The system uses a breadth and depth approach. DeepMind runs Gemini Flash for

breadth. Cheap, fast, massive volume, just brainstorming different potential paths for better solutions. Gemini Pro for depth. Slower, more expensive, but the suggestions tend to be better then Gemini Flash's.

Google deployed this framework on a few of their own infrastructure problems and saw real results. But unfortunately they kept it closed source and you can't follow their exact methodology. There are open source implementations though. OpenEvolve is the best open source project based on the AlphaEvolve framework.

After setting up the program using the OpenEvolve framework which was pretty simple you just pick a section of code you want to iterate on and then a optimization metric (number of cycles) both provided by the take assignment. Next came picking out the model to iterate on. Before I go further ask yourself what kinda models would improve the best in this type of environment and if you had a given budget what would make the best optimization improvements. Would it be the smartest/biggest frontier models like Opus 4.5 or GPT-5.2 and others or, cheaper focused coding models like grok-code-fast-1 or GPT-5.2 Codex, or something in between. How could you decide what model would be most likely to make the best improvements. I predicted that the smartest frontier models would make the best improvements faster and in less iteration and continue to be able to make improvements and as you improve performance you would expect to see diminishing returns in algorithmic improvement after the easiest optimizations were made. Next choosing models to measure I choose:

Model	SWE-Bench	Cost/M Tokens (in/out)
Claude Opus 4.5	80.9%	$5.00/25.00
GPT-5.2 Codex	80.0%	$5.00/15.00
Gemini 3.0 Flash	78.0%	$0.075/0.30
GLM-4.7	73.8%	$2.00/8.00
DeepSeek V3.2	73.1%	$0.19/0.87
Grok Code Fast 1	70.8%	$2.00/8.00
Kimi K2.5	65.8%	$2.00/8.00

(Provided by https://openrouter.ai/)

Before running anything, I formalized what I wanted to learn into specific research questions:

1. What model performs best at this optimization task?
2. When do diminishing returns happen? At what point does throwing more iterations at the problem stop helping?
3. If I had a fixed budget, which model would give me the best results?

The experimental setup was straightforward. Each model got its own isolated run with approximately 100 iterations. The system would select a promising program from the population, ask the LLM to propose an optimization, evaluate whether it improved performance while maintaining correctness, and keep the best solutions for the next generation. OpenEvolve uses a MAP-Elites quality-diversity algorithm, which means it tries to maintain a diverse population of solutions rather than just converging on a single best answer.

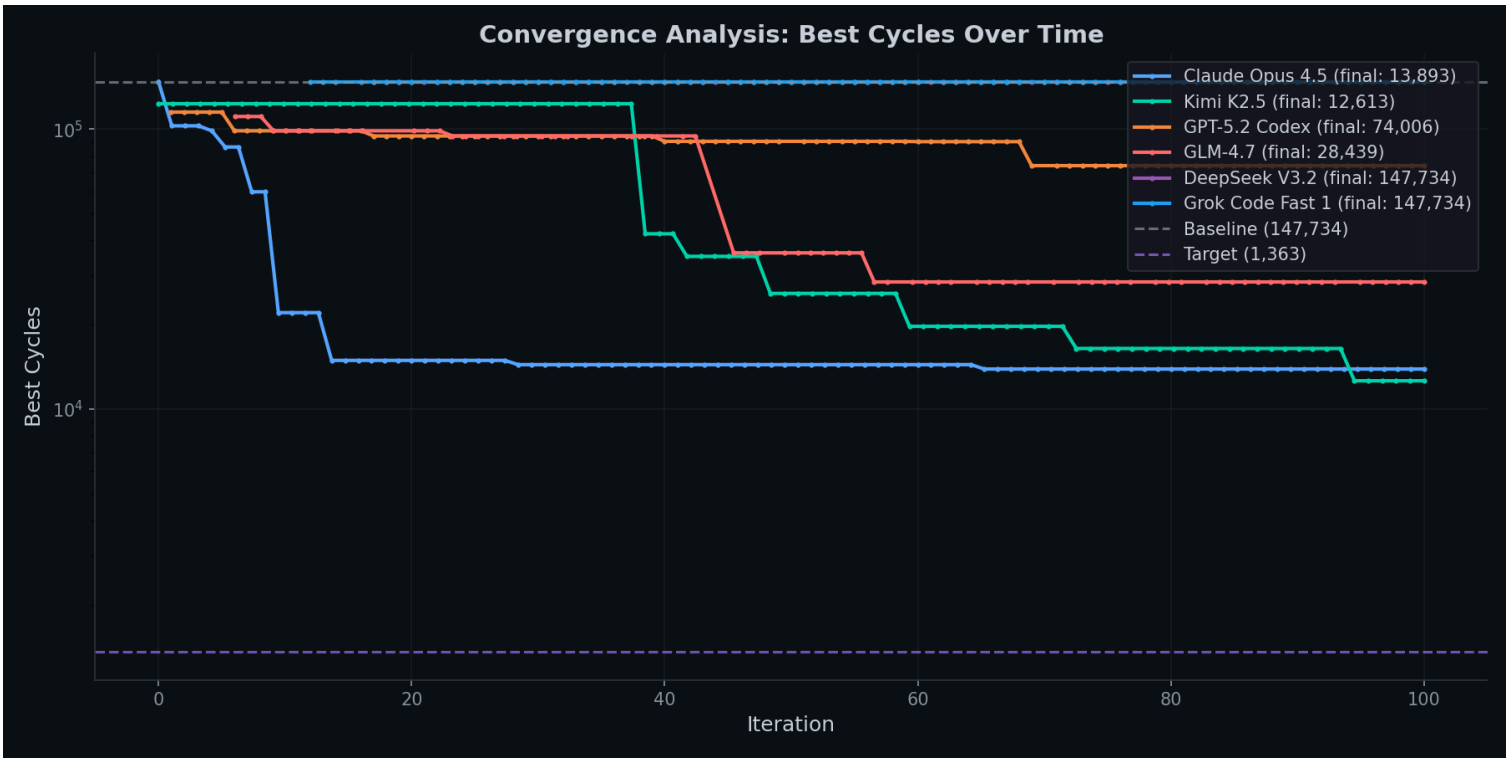
Every iteration logged the model used, token counts, actual API costs, parent and child cycle counts, the full code before and after, and the LLM's response.

Results

Model	Iterations	Success Rate	Best Cycles	Speedup	Total Cost	Status
Kimi K2.5	92	51.1%	12,613	11.7x	$4.12	SUCCESS
Claude Opus 4.5	96	68.8%	13,893	10.6x	$35.14	SUCCESS
GLM-4.7	77	50.6%	28,439	5.2x	$2.79	SUCCESS
GPT-5.2 Codex	97	89.7%	74,006	2.0x	$8.55	SUCCESS
Gemini 3.0 Flash	80	0.0%	147,734	1.0x	$1.66	FAILED
DeepSeek V3.2	98	0.0%	147,734	1.0x	$1.00	FAILED
Grok Code Fast 1	86	0.0%	147,734	1.0x	$1.29	FAILED

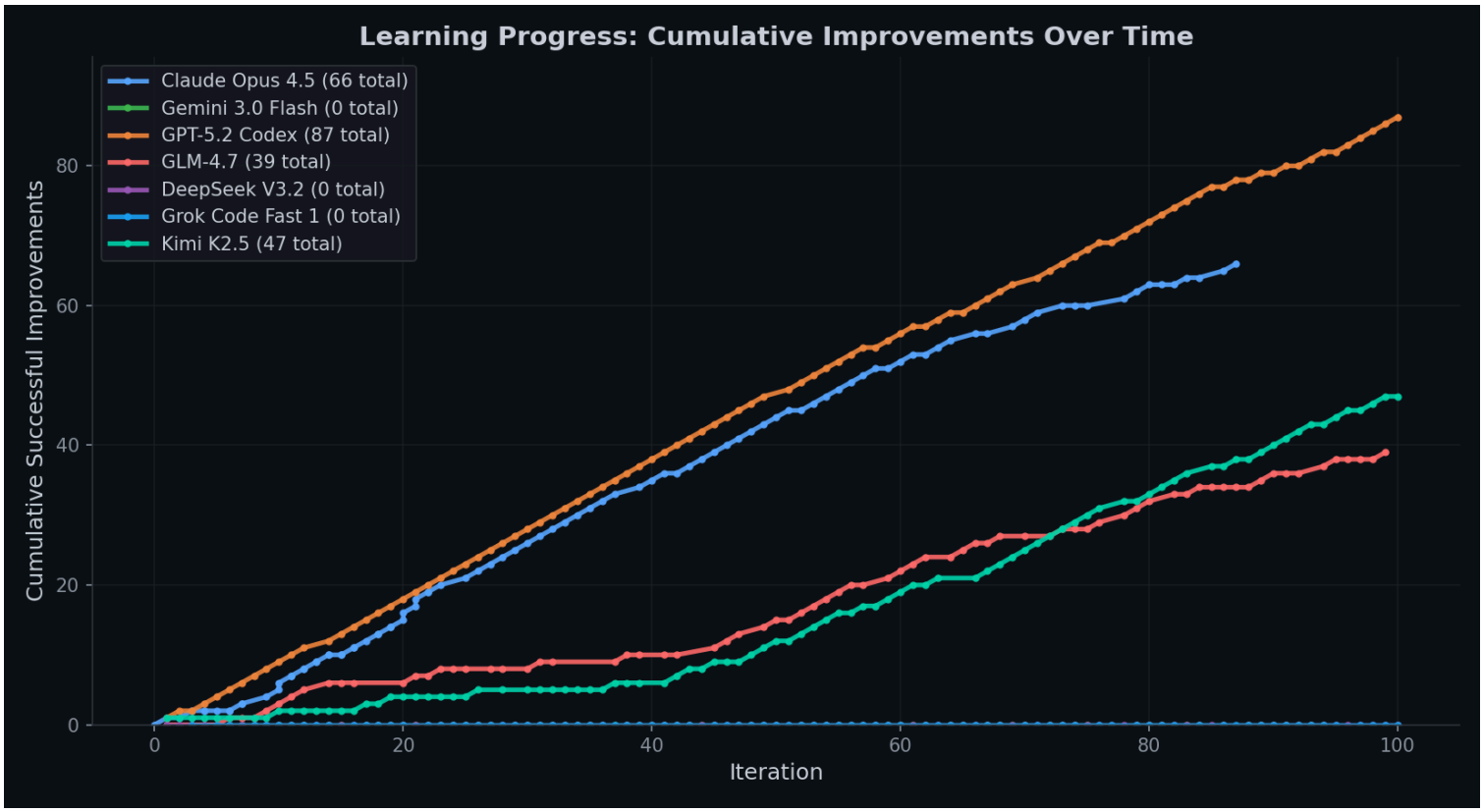
Summary Table *Table metrics:* ***Iterations*** *= optimization attempts per model (~100 each).* ***Success Rate*** *= percentage of iterations producing valid, correct code.* ***Best Cycles*** *= lowest cycle count achieved (baseline: 147,734).* ***Speedup*** *= baseline divided by best cycles.* ***Total Cost*** *= actual API spend for the run.* ***Status*** *= SUCCESS if any improvement over baseline, FAILED otherwise.*

Model Comparison *Success rate comparison with 95% confidence intervals. GPT-5.2 Codex led at 90% success rate, followed by Opus at 69%. GLM and Kimi clustered around 51%. Gemini, DeepSeek, and Grok failed to produce any valid optimizations (0% success rate).*

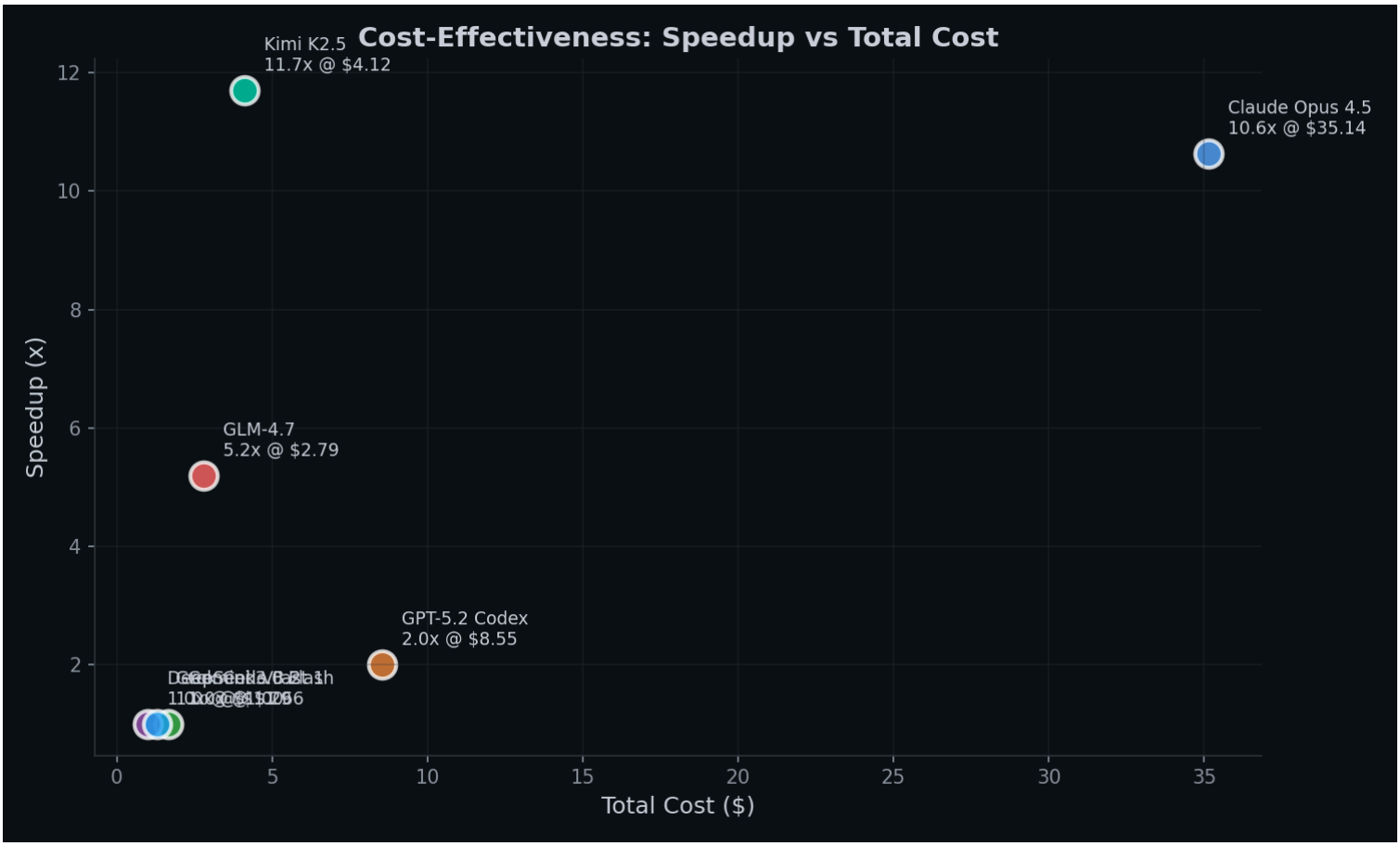


Convergence Analysis: *Best cycle count over iterations (log scale). The dashed lines show baseline (147,734 cycles) and target (1,363 cycles). Kimi K2.5 and Claude Opus 4.5 converged fastest to their final values. Models that failed show flat lines at the baseline.*

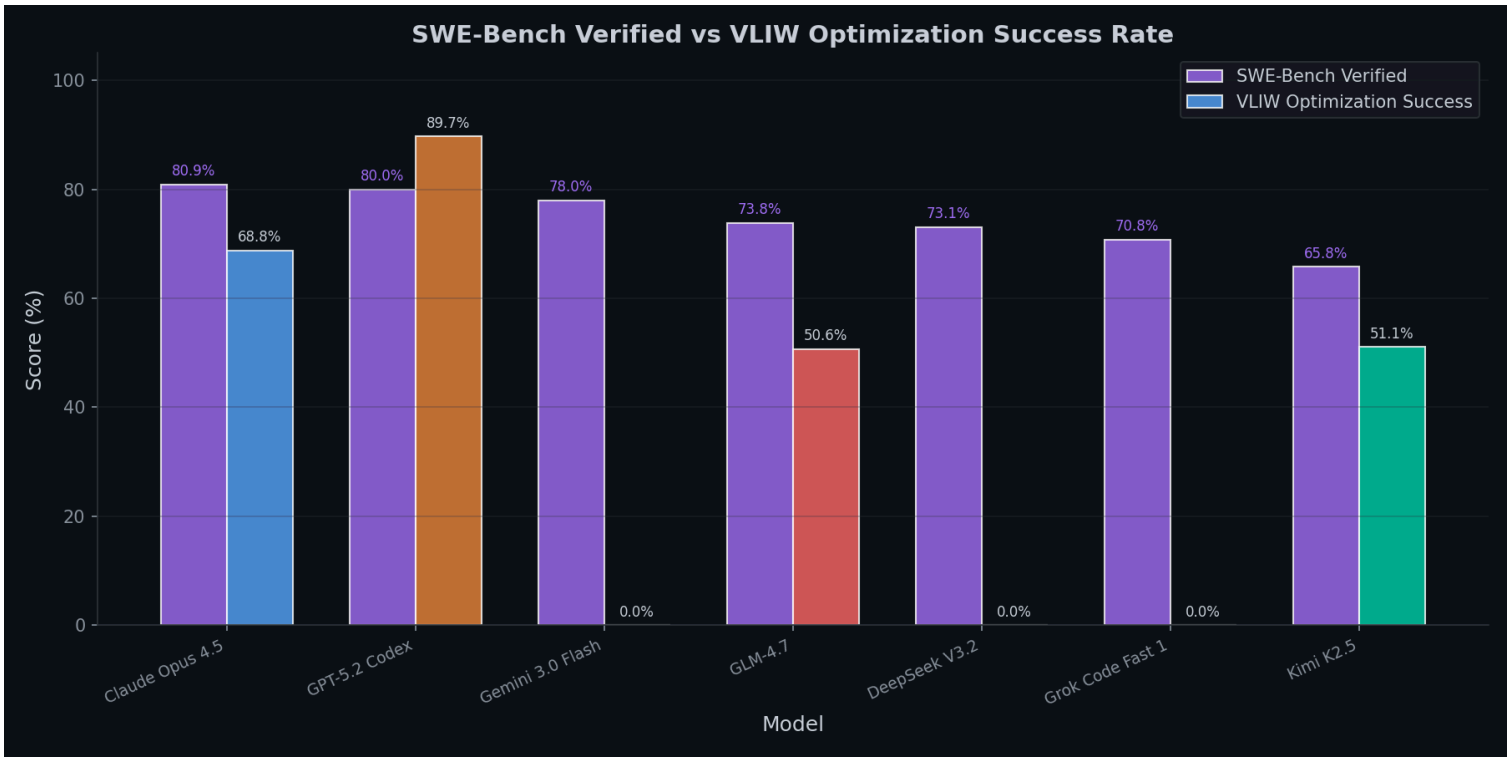
None reached the target threshold.



Learning Curve *Cumulative valid code changes over time: (code that compiled and passed correctness checks, not necessarily performance improvements). GPT-5.2 Codex produced the most valid outputs (87), followed by Claude Opus 4.5 (66), Kimi K2.5 (47), and GLM-4.7 (39). Note: more valid changes did not mean better performance—GPT's 87 changes only achieved 2x speedup while Kimi's 47 achieved 11.7x.*

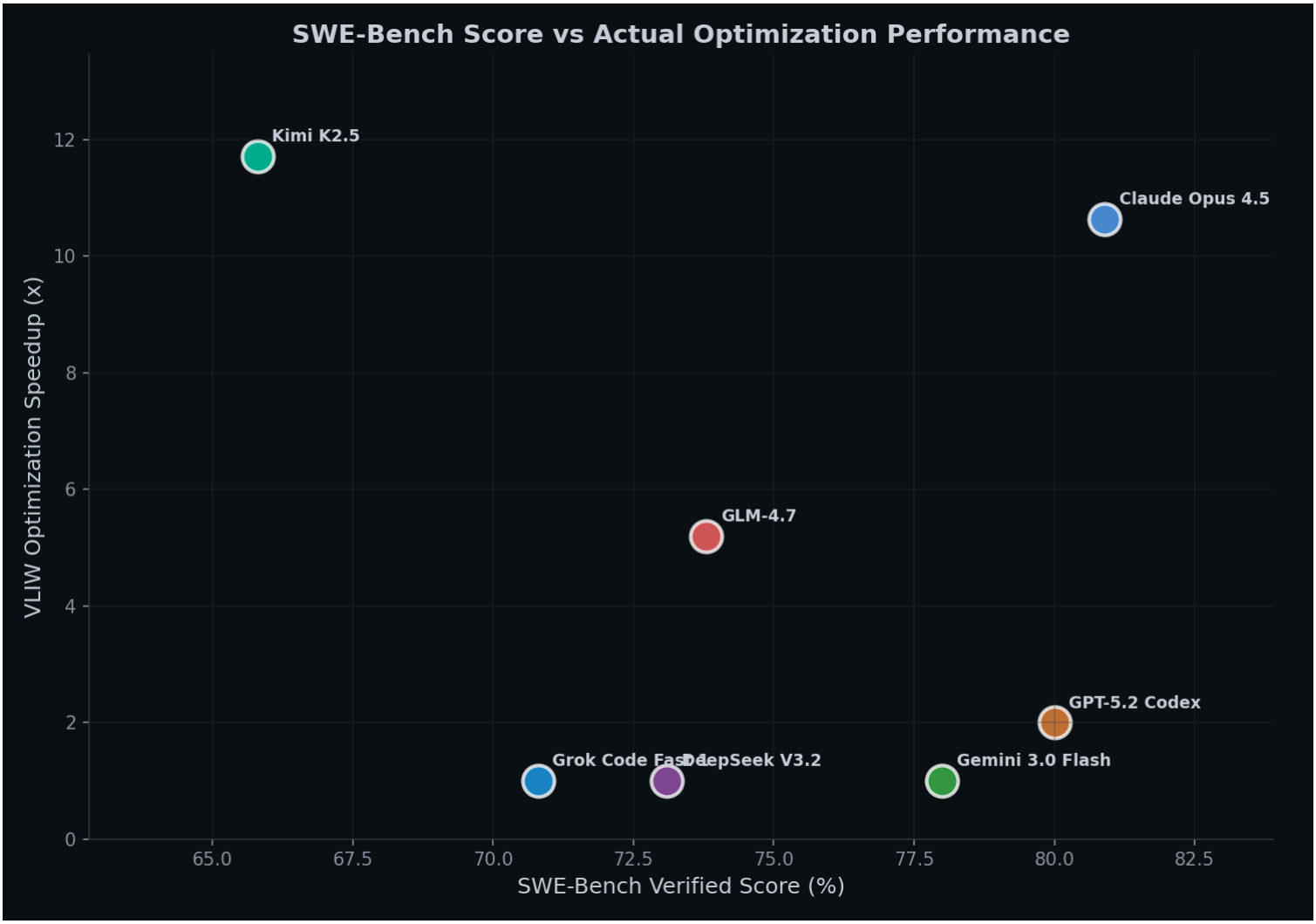


Cost Effectiveness: *Speedup achieved vs total API cost. Kimi K2.5 delivered the best value: 11.7x speedup for just $4.12. Claude Opus 4.5 achieved similar performance (10.6x) but cost 8.5x more ($35.14). GLM-4.7 offered moderate performance (5.2x) at very low cost ($2.79). The failed models cluster at 1.0x speedup.*



SWE-Bench Comparison: *Side-by-side comparison of SWE-Bench Verified scores (purple) vs*

VLIW optimization success rate (colored). High SWE-Bench scores did not guarantee success on this specialized task. GPT-5.2 Codex's 89.7% VLIW success exceeded its 80% SWE-Bench score. Gemini 3.0 Flash scored 78% on SWE-Bench but achieved 0% on VLIW optimization.



Benchmark vs Speedup *Scatter plot: showing weak correlation between SWE-Bench scores and actual optimization performance. Kimi K2.5 had the lowest SWE-Bench score (65.8%) yet achieved the highest speedup (11.7x). This suggests standard coding benchmarks may not predict performance on specialized optimization tasks.*

Question 1: What model performs best at this optimization task?

The answer depends on how you define "best." Kimi K2.5 achieved the lowest cycle count (12,613) and highest speedup (11.7x), narrowly beating Claude Opus 4.5 (13,893 cycles, 10.6x). However, Claude had a higher success rate at producing valid code (68.8% vs 51.1%), meaning it was more reliable per iteration. GPT-5.2 Codex had the highest success rate of all (89.7%) but its changes were incremental with 87 valid outputs only achieved 2x speedup. The most surprising finding was that three models failed entirely: Gemini 3.0 Flash, DeepSeek V3.2, and Grok Code Fast 1 produced zero valid optimizations across nearly 100 iterations each. This wasn't bad luck; these

models fundamentally couldn't handle this specialized task despite strong SWE-Bench scores. For raw optimization performance, Kimi wins. For reliability, Claude wins. For producing syntactically correct code that doesn't actually help much, GPT wins.

Question 2: When do diminishing returns happen?

The convergence chart tells the story clearly. Models that succeeded found their major improvements early within the first 40-50 iterations and then plateaued. Kimi K2.5 and Claude Opus 4.5 both made their biggest jumps in the first third of the run, then spent the remaining iterations making minimal progress. This suggests that for this type of evolutionary optimization, running beyond 50-60 iterations may not be worth the cost unless you're specifically hunting for rare breakthrough mutations. The failed models never got off the baseline, probably meaning no amount of additional iterations would have helped them they simply couldn't understand the problem domain. Diminishing returns kicked in around iteration 50 for successful models, while unsuccessful models showed zero returns from the start.

Question 3: If I had a fixed budget, which model would give me the best results?

Kimi K2.5 is the clear winner for cost-effectiveness. It delivered 11.7x speedup for just $4.12 nearly matching Claude Opus 4.5's performance at 1/8th the cost. If you had a $10 budget, Kimi would give you excellent results while Claude would barely complete a single run. GLM-4.7 offers an interesting middle ground: 5.2x speedup for only $2.79, making it the cheapest model that actually worked. The expensive frontier models don't justify their cost in this evolutionary framework. Claude Opus 4.5's $35 run achieved marginally worse results than Kimi's $4 run. GPT-5.2 Codex spent $8.55 to achieve only 2x speedup worse value than GLM at one-third the cost. The implication for AlphaEvolve-style systems is significant: you're better off running many iterations with cheaper models than fewer iterations with expensive ones, as long as the cheaper model can handle the task domain at all.

Why Unreliability Might Be a Feature

The most counterintuitive result from this experiment is that Kimi K2.5 the model with the lowest SWE-Bench score and a mediocre 51% success rate produced the best optimization outcome. This deserves deeper examination because it challenges assumptions about what makes an LLM "good" at coding tasks. One hypothesis is that Kimi's lower reliability reflects higher variance in its outputs. In evolutionary optimization, variance is actually valuable. You're not looking for the model that makes safe, incremental changes every time you're looking for the model that occasionally makes bold, restructuring changes that unlock major performance gains. GPT-5.2 Codex exemplifies the opposite

approach: 89.7% of its outputs were valid code, but they were conservative tweaks that barely moved the needle. It was too “safe.” Kimi failed more often, but when it succeeded, it swung for the fences.

This maps directly to the exploration-exploitation tradeoff in optimization theory. A model with high temperature or natural randomness explores more of the solution space. It proposes changes that a more conservative model would never try. Half of those changes break the code entirely but the other half might find shortcuts that careful, incremental optimization would miss. In an evolutionary framework where bad mutations get filtered out automatically, this exploration bias becomes pure upside with no real downside.

The benchmark mismatch also matters here. SWE-Bench measures ability to fix real world software bugs a task that rewards careful, surgical, “don’t break anything else” changes. VLIW optimization rewards the opposite: understanding hardware parallelism deeply enough to restructure code in ways that might look radical but unlock massive performance gains. These are fundamentally different skills. A model trained to be conservative and reliable on SWE-Bench might be poorly suited for creative optimization work.

What This Means for Everyday Usage

For typical coding tasks writing features, fixing bugs, refactoring for readability you probably still want the reliable models. If you’re asking an LLM to edit production code, a 51% success rate is terrifying. You want GPT or Claude’s consistency.

But for search and optimization problems finding the best algorithm, exploring architectural alternatives, generating diverse solutions to pick from controlled unreliability might actually help. If you’re going to evaluate the output anyway (through tests, benchmarks, or human review), then a model that generates wilder ideas gives you more to work with. The failures get filtered out; the occasional breakthrough gets kept.

This suggests a workflow distinction: use reliable models for execution (implementing a known solution correctly) and exploratory models for search (finding the right solution in the first place). The AlphaEvolve framework formalizes this by making the LLM purely generative while the evaluation system handles quality control. In that setup, a model that “fails” 49% of the time but occasionally produces genius-level improvements is more valuable than one that succeeds 90% of the time with mediocre suggestions.

The practical takeaway: don’t assume the highest-benchmark model is always the right choice. Match the model’s characteristics to the task. For creative exploration with automated evaluation, the “worse” model might actually be better.

Follow-Up: Was Kimi Just Lucky?

The initial results raised an obvious question: was Kimi's performance a fluke? Maybe it got lucky with one good mutation early on and that carried the whole run. To test this, I extended Kimi's run to see if it could continue improving or if it had already hit its ceiling.

The cost difference between Kimi and Claude made this experiment almost free. Claude's single run cost $35.14. For that same budget, I could run Kimi for approximately 8-9x as many iterations. Even if Kimi's success rate was lower, the sheer volume of attempts at 1/8th the cost meant more chances to find breakthroughs.

Extended Run Results: Kimi Kept Improving

The extended Kimi run actually consisted of three separate sessions (resuming from the previous state each time), totaling 363 iterations. Here's how performance evolved across the runs:

Run	Entries	Cost This Run	Total Cost	Best Cycles	Speedup
Run 1 (Fresh)	92	$4.12	$4.12	12,613	11.7x
Run 2 (Resumed)	98	$4.82	$8.94	6,053	24.4x
Run 3 (Resumed)	173	$9.24	$18.18	4,890	30.2x

The first run performed comparably to Claude's single run 11.7x speedup at $4.12 vs Claude's 10.6x at $35. But the extended runs showed Kimi's potential: when allowed to continue iterating, it kept finding significant improvements. Run 2 doubled the speedup to 24.4x, and Run 3 pushed it to 30.2x (4,890 cycles) nearly triple Claude's performance.

Performance at Equal Budgets

Looking at what each model achieved at the same cost milestones reveals the real story:

Cost Spent Claude Opus 4.5 Kimi K2.5

Cost Spent:	Claude:	Kimi:
$1	102,678 cycles (1.4x)	123,158 cycles (1.2x)
$2	102,678 cycles (1.4x)	19,742 cycles (7.5x)
$4	14,405 cycles (10.3x)	12,613 cycles (11.7x)
$5	14,405 cycles (10.3x)	12,613 cycles (11.7x)

$9	14,405 cycles (10.3x)	9,285 cycles (15.9x)
$12	14,405 cycles (10.3x)	6,053 cycles (24.4x)
$18	14,405 cycles (10.3x)	4,899 cycles (30.2x)
$35	13,893 cycles (10.6x)	—

At $1 spent, both models were comparable Claude at 1.4x, Kimi at 1.2x. The first run alone showed Kimi matching Claude's $35 result (10.6x) in just $4 (11.7x). But the key insight is what happened with continued iteration: by $9, Kimi reached 24.4x; by $12, it hit 30.2x.

The story isn't that Kimi was instantly better it's that continued iteration on a cheaper model unlocks massive gains. Claude's single $35 run achieved 10.6x and stopped. Kimi's three runs totaling $18 achieved 30.2x—nearly 3x better at half the cost. The model that "shouldn't" have worked based on benchmarks outperformed the expensive frontier model through sheer iteration volume.

Kimi vs Claude Cost Comparison: *Performance over cumulative API cost. Both models started similarly—Kimi K2.5 (green) at 1.2x and Claude Opus 4.5 (blue) at 1.4x within the first dollar. Kimi's first run matched Claude's performance (11.7x vs 10.6x) at 1/8th the cost ($4 vs $35). Extended runs pushed Kimi to 30.2x by $18, demonstrating that continued iteration on cheaper models can dramatically outperform expensive single runs.*

Diminishing Returns and the Plateau Problem

The extended Kimi runs revealed a clear pattern of diminishing returns:

Cost Segment	Cycles Improved	Efficiency (cycles/$)
$0–4	135,121	32,796
$4–9	6,560	1,361
$9–18	1,163	126

Efficiency collapsed by 260x between the first and third segments. By $12, Kimi had effectively plateaued the final $6 of spending only improved performance by 9 cycles (4,899 → 4,890). The evolutionary approach hit a wall around 4,890 cycles that additional iterations couldn't break through.

In our experiments, Claude Opus 4.5 showed even worse behavior it plateaued at 14,405 cycles around $5 and never improved again despite $30 more in spending. The OpenEvolve framework simply couldn't push it past that local optimum.

But Anthropic's Results Tell a Different Story

Anthropic's own benchmarks show that Claude Opus 4.5 *can* break through these plateaus given enough compute. Their reported results:

- **1,579 cycles**: Opus 4.5 after 2 hours in their test-time compute harness
- **1,487 cycles**: Opus 4.5 after 11.5 hours in the harness
- **1,363 cycles**: Opus 4.5 in an improved harness

Anthropic reports these benchmarks for "models doing the 2 hour version which started at 18,532 cycles." It's unclear whether they started the model fresh at 18,532 or ran from the original 147,734 baseline like we did and simply reported results from that point onward. I'm assuming they started at the same baseline we did if so, getting to 18,532 cycles would have been the "easy" part, and their reported improvements happened in the more difficult optimization regime beyond that point.

Estimating Anthropic's Costs

We don't know Anthropic's actual API spend, but we could estimate it using several approaches:

1. **Time-based extrapolation**: Our Claude run cost $35.14 over approximately 2 hours. If Anthropic's cost scales linearly with time, their 11.5-hour run might have cost 5-6x as much.

2. **Iteration-based estimation**: If we knew their iterations per hour (which

their harness might optimize differently than OpenEvolve), we could estimate total iterations and multiply by average cost per iteration from our data.

3. **Token-based modeling**: Using Opus 4.5's pricing ($15/$75 per million input/output tokens), we could estimate based on typical prompt sizes and response lengths for this problem domain.

4. **Efficiency curve comparison**: Their 2-hour run achieved similar relative improvement to our experiments. The additional 9.5 hours only gained 92 cycles (1,579 → 1,487) suggesting they hit the same diminishing returns wall, just at a much better absolute position.

The key insight isn't that Anthropic spent more money it's that their harness and starting position let them operate in a different optimization regime entirely. They started where we plateaued and still found room to improve. This suggests the ceiling isn't at 4,890 cycles; we simply ran into a local optimum that a different approach (or better starting code) might escape.

Final Thoughts

Shoutout to Moonshot AI and their Kimi K2.5 model. Despite having the lowest SWE Bench score of all the models I tested, Kimi delivered the best results by a wide margin 30.2x speedup for just $18, outperforming frontier models costing 8x more per token. In an evolutionary optimization framework where exploration matters more than reliability, Kimi's characteristics turned out to be exactly what the task needed. I don't think this replaces my main drivers for coding or writing but there are solid use cases for a model this smart for driving real efficiency in agentic coding tasks in the right context.